

Exercício 12 — Identificar CWEs

Identifique e classifique os CWEs presentes no trecho de código abaixo:

```
import sqlite3
def get_user(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username='" + username + "';"
    cursor.execute(query)
    user = cursor.fetchone()
    conn.close()
    return user
```

Classifique usando a taxonomia CWE, discuta as implicações de segurança e recomende modificações para mitigar as vulnerabilidades.

12. Identificar CWEs

Código em Análise

```
import sqlite3

def get_user(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username='" + username + "';"
    cursor.execute(query)
    user = cursor.fetchone()
    conn.close()
    return user
```

Passo 1 — Identificar o Problema

Observe esta linha:

```
query = "SELECT * FROM users WHERE username='" + username + "';"
```

O valor de `username` vem diretamente do chamador — e é **colado na string SQL sem nenhuma sanitização**.

Se um invasor passar isso como `username`:

```
' OR '1'='1
```

A consulta se torna:

```
SELECT * FROM users WHERE username='' OR '1'='1';
```

`'1'='1'` é sempre verdadeiro → **o banco de dados retorna todos os usuários da tabela**.

Ou com esta entrada:

```
admin'; DROP TABLE users; --
```

A consulta se torna:

```
SELECT * FROM users WHERE username='admin'; DROP TABLE users; --';
```

Toda a tabela `users` é **excluída**.

Passo 2 — Classificar os CWEs

Principal: CWE-89 — Injeção de SQL

Campo	Detalhe
ID CWE	CWE-89
Nome	Neutralização Imprópria de Elementos Especiais usados em um Comando SQL

Campo	Detalhe
Nível na Taxonomia	Base
Pai	CWE-943: Neutralização Imprópria de Elementos Especiais em Consulta de Dados
Top 25	Sim — consistentemente no top 3

Causa raiz: A entrada do usuário é concatenada diretamente em uma consulta SQL, em vez de ser tratada como dado.

Contribuinte: CWE-20 — Validação Inadequada de Entrada

Campo	Detalhe
ID CWE	CWE-20
Nome	Validação Inadequada de Entrada
Nível na Taxonomia	Classe
Papel aqui	Nenhuma validação do <code>username</code> antes do uso — qualquer string é aceita

Adicional: CWE-200 — Exposição de Informações Sensíveis

Campo	Detalhe
ID CWE	CWE-200
Nome	Exposição de Informações Sensíveis a um Ator Não Autorizado
Papel aqui	<code>SELECT *</code> retorna todas as colunas — incluindo senhas, tokens ou qualquer campo sensível

Passo 3 — Implicações de Segurança e Cenários de Ataque

Cenário 1 — Bypass de autenticação

```
username = "' OR '1'='1"
```

Retorna todos os usuários. Se a aplicação verifica `if user: conceder_acesso()`, o invasor faz login como o primeiro usuário da tabela — frequentemente um administrador.

Cenário 2 — Exfiltração de dados

```
username = "' UNION SELECT username, password, null FROM users --"
```

Despeja todos os nomes de usuário e senhas do banco de dados.

Cenário 3 — Destruição de dados

```
username = "'; DROP TABLE users; --"
```

Exclui toda a tabela de usuários. Sem recuperação sem backup.

Cenário 4 — Comprometimento total do servidor

Em alguns bancos de dados (MySQL com privilégio FILE, SQL Server com xp_cmdshell), a Injeção de SQL pode levar à **execução de comandos do sistema operacional**.

Passo 4 — Corrigir o Código

A correção: use consultas parametrizadas

```
import sqlite3

def get_user(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = "SELECT id, username FROM users WHERE username = ?;" # sem colunas
```

```

sensíveis
    cursor.execute(query, (username,)) # username
passado como dado
    user = cursor.fetchone()
    conn.close()
    return user

```

O que mudou e por quê:

Mudança	Motivo
Placeholder <code>?</code> em vez de concatenação	O banco de dados trata <code>username</code> como dado , nunca como código SQL — injeção é impossível
<code>(username,)</code> como segundo argumento	O driver cuida do escape — o desenvolvedor não precisa fazer isso manualmente
<code>SELECT id, username</code> em vez de <code>SELECT *</code>	Retorna apenas o necessário — senhas e campos sensíveis não são expostos (corrige CWE-200)

Opcional — adicionar validação de entrada (defesa em profundidade)

```

import re

def get_user(username):
    if not username or len(username) > 50:
        return None
    if not re.match(r'^[a-zA-Z0-9_]+$', username): # apenas caracteres seguros
        return None
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = "SELECT id, username FROM users WHERE username = ?;"
    cursor.execute(query, (username,))
    user = cursor.fetchone()
    conn.close()
    return user

```

A validação sozinha **não** corrige a Injeção de SQL — as consultas parametrizadas corrigem. A validação é uma camada extra (corrige CWE-20).

Resumo

CWE	Severidade	Correção
CWE-89 Injeção de SQL	Crítico	Use consultas parametrizadas — sempre
CWE-20 Validação de Entrada	Médio	Valide comprimento e caracteres permitidos
CWE-200 Exposição de Informações	Médio	Use <code>SELECT coluna</code> em vez de <code>SELECT *</code>

*A lição principal: **nunca construa consultas SQL com concatenação de strings**. A entrada do usuário é dado — deixe o driver do banco de dados tratá-la como dado, nunca como código.*